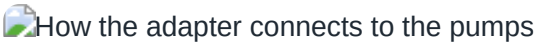
pondpump logo

ioBroker.pondpump

User Manual — set up, control and monitor your OASE AquaMax Eco Titanium pond pumps in ioBroker

🐟 **Beginner-friendly guide**

1. What this adapter does



How ioBroker reaches your pumps: today via the OASE cloud, and — in local mode — directly over your LAN.

The **pondpump** adapter connects ioBroker to your **OASE AquaMax Eco Titanium** pond pump(s) through the **OASE Garden Controller Cloud (EGC)** gateway. Once it is running you can, from ioBroker (and therefore from VIS, scripts, scenes, Alexa, etc.):

- **switch each pump on and off**,
- **set the pump speed** from 0 – 100 %,
- **read live telemetry**: power (W), motor speed (rpm), water/electronics temperature (°C) and mains voltage (V),
- **see the connection and device status**.

Each pump keeps the name you gave it in the OASE app (e.g. *Waterfall*, *Filter*), so you can recognise it in the object tree.

> 💡 **Good to know:** the adapter talks to the **controller** (item 55317), which in turn talks to the pumps (item 73656). It is a separate project from the community adapter for the OASE socket controllers — it targets the smart pond pumps.

---

2. Before you start — what you need

You need	Why
A running **ioBroker** installation (js-controller, Node.js ≥ 22)	The platform this adapter runs on
An **OASE Garden Controller Cloud** (EGC, item 55317), set up in the OASE app	The gateway the adapter connects to
One or two **OASE AquaMax Eco Titanium** pumps (item 73656), paired in the app	The devices being controlled
Your pumps **already working in the OASE app**	The adapter uses the same cloud account
A **cloud refresh token** (see chapter 4)	How the adapter logs in without your password

> 💡 **Tip:** get everything working in the **OASE app first**. If the app can switch the pumps, the adapter can too.

---

3. Installing the adapter

The adapter is published on npm as `iobroker.pondpump`. Until it is part of the official ioBroker repository, install it from its source:

1. Open the ioBroker **Admin** UI.
2. Go to **Adapters** and switch on **Expert mode** (the wizard-hat icon, top right).
3. Click the **cat/octocat "Install from own URL"** icon.
4. Enter one of:
   - the npm package name `iobroker.pondpump`, or
   - the GitHub URL of a release tarball if you were given one.
5. Confirm and wait until the adapter appears in the list.
6. Click the **+** on the pondpump tile to create an **instance** (`pondpump.0`).

The instance configuration opens automatically. Leave it for a moment — first we need a refresh token (next chapter).

---

4. Getting a cloud refresh token (step by step with mitmproxy)

The OASE cloud uses **Microsoft Azure AD B2C** for login. For security the adapter does **not** store your account password. Instead it uses a **refresh token** — a long, one-time credential that your OASE app receives when it logs in. You capture that token **once** with a small tool called **mitmproxy**, paste it into the adapter, and from then on the adapter refreshes it automatically.

Don't worry if you've never done this — follow the steps below exactly.

mitmproxy sits between your phone and the OASE cloud, so you can read the login and copy the token.

4.1 Install and start mitmproxy

mitmproxy is a small, free program. We'll use its browser version, **mitmweb**. Follow the steps for **your** operating system.

Windows (using PowerShell)

1. Open your web browser and go to **https://mitmproxy.org/downloads/**.
2. Download the **Windows** package (the newest version — usually a `.msi` installer).
3. Open the downloaded file and click through the installer: **Next** → **Next** → **Install** → **Finish**.
4. Now open **PowerShell**:
   - Press the **Windows key**, type `PowerShell`, and click **Windows PowerShell** in the list.
   - A dark window with a blinking text cursor appears — this is the command line.
5. Type this command and press **Enter** (it pins the proxy port to **8080**):

   ```
   mitmweb --listen-port 8080
   ```

6. If Windows asks whether to **allow network access**, click **Allow**. The control panel opens automatically at **http://127.0.0.1:8081** in your computer's browser — that's the mitmproxy panel you'll watch. The actual **proxy listens on port 8080** and waits there for the phone's traffic. ✅
7. **Keep this PowerShell window open** the whole time — closing it stops mitmproxy. Later, to stop it, click the window and press **Ctrl + C**.

> 💡 **"mitmweb is not recognized"?** Close PowerShell and open it again (so it notices the newly installed program). If you downloaded the `.zip` version instead of the installer, unzip it, then in PowerShell type `cd` followed by the folder path and run `.\mitmweb.exe`.

macOS

1. Open **Terminal** (press **Cmd + Space**, type `Terminal`, press Enter).
2. The easiest way is with Homebrew: run `brew install mitmproxy`. (No Homebrew? Download the macOS build from **https://mitmproxy.org/downloads/** and unzip it.)
3. Run `mitmweb --listen-port 8080`. A browser tab opens at **http://127.0.0.1:8081**.

Linux

1. Install it with `pipx install mitmproxy` (or your distribution's package, or the binaries from the downloads page).
2. Run `mitmweb --listen-port 8080` in a terminal and open **http://127.0.0.1:8081**.

In every case: the browser page on **:8081** is the control panel you'll watch, and **port 8080** is where your phone will send its traffic (next step).

4.2 Send your phone's traffic through mitmproxy

Your phone and computer must be on the **same Wi-Fi**.

1. Find your **computer's local IP** (e.g. `192.168.1.20`): Windows `ipconfig`, macOS/Linux `ip addr` / `ifconfig`.
2. On the phone: **Wi-Fi settings** → **your network** → **Proxy** → **Manual** and enter **Server = your computer's IP**, **Port = 8080**. Save.
3. Open the phone's browser and visit **http://mitm.it**. Choose your phone's system, **install** the offered certificate **and trust it**:
   - **iOS:** install the profile, then *Settings* → *General* → *About* → *Certificate Trust Settings* and turn the mitmproxy certificate **on**.
   - **Android:** install it as a **CA certificate** (Settings → Security → Encryption & credentials → Install a certificate → CA certificate).

   This certificate is what lets mitmproxy read the otherwise-encrypted OASE traffic. **Remove the proxy and the certificate again when you're done.**

4.3 Capture the login and grab the refresh token

1. In the **mitmweb** page, clear the list (so new requests are easy to spot).
2. In the **OASE app**: **log out**, then **log back in**.
3. In mitmweb's **filter box** at the top, type one of these to jump straight to the right request — this is the trick that saves you scrolling through hundreds of entries:

Type this filter	It shows
`~u token`	only requests whose URL contains "token"
`~d account.oase.com`	only requests to the OASE login server
`~b refresh_token`	only requests whose body contains `refresh_token`

4. Click the **POST** request that ends in `/oauth2/v2.0/token`.
5. Open its **Request** tab and look at the form body. Find `refresh_token=` and copy the long value after it (up to the next `&`).
   - **Extra tip:** press `/` in mitmweb and search for `refresh_token` to highlight it instantly.
6. Paste that value into the adapter setting **"Cloud refresh token"** (chapter 5).

> 💡 The refresh token is long (hundreds of characters) — copy **all** of it. Treat it like a password: don't share it. You can revoke it any time by logging out everywhere in the OASE app. **Your account password is never entered into the adapter.**

4.4 (Advanced) Find the device password for local mode

Only needed if you want connection mode `local` (chapter 8). While mitmproxy is still running:

1. In the app, open your pond so it loads the pumps (this triggers the inventory download).
2. In mitmweb's filter box, type `~u Inventory` to show the request to `/User/Inventory`.
3. Click it and open the **Response** tab. In the JSON, find the pump's **custom attributes**; the entry with `Id` **= 101** holds the **device password** — a **64-character** value (it may contain `\uXXXX` escape sequences, that's fine).
4. Copy that value into the adapter setting **"Device password"**. The adapter decodes it and uses it for the local TLS handshake.

> 💡 **If mitm.it won't load:** double-check the phone's proxy points at your computer's IP on port 8080, and that traffic is flowing. On iOS you must both **install** *and* **trust** the certificate (two separate steps).

---

5. Configuring the instance

Open **Instances** → **pondpump.0** → **settings** (the wrench icon). The settings are grouped:

Connection

Setting	What to enter
Connection mode	`cloud` for the internet path, `local` for the in-house LAN path (chapter 8). The two are mutually exclusive.
Poll interval	How often (seconds) the adapter reads status. Default **30**. Minimum 5.

Cloud

Setting	What to enter
Cloud refresh token	The token from chapter 4 (stored encrypted).
Advanced (base URL, token URL, client id, scope)	Leave at their defaults unless OASE changes their cloud.

Local (only for `local`)

Setting	What to enter
Controller IP	The EGC controller's address in your network.
Device password	The 64-character device password (advanced; see chapter 8).
Bind address / Port	The ioBroker host address and TCP port the controller connects back to (default 5999).

Click **Save**. The instance starts and, after a few seconds, `info.connection` should turn **true**.

---

6. The objects the adapter creates

After the first successful poll you will find these under `pondpump.0`:

```
pondpump.0
├── info.connection            (true when connected)
├── <gateway>                  the EGC controller (device)
│   ├── serialNumber, firmware
│   └── online                 (controller reachable)
└── pumps.<deviceNumber>       one device per pump, named after the app
    ├── control.on             ← switch on/off        (writable)
    ├── control.speed          ← speed 0–100 %        (writable)
    ├── control.speedRaw       ← speed 0–255 (raw)    (writable)
    ├── status.connected       pump reachable
    ├── status.fcStatus        controller status text
    └── telemetry
        ├── power              live power in W
        ├── speed              live motor speed in rpm
        ├── temperature        °C
        ├── temperature2       °C (second sensor)
        ├── voltage            mains voltage in V
        └── raw.sensorN        still-unclassified sensor values
```

---

7. Controlling and reading the pumps

Switch a pump on/off — set `pumps.<deviceNumber>.control.on` to `true` / `false`.

Set the speed — write a percentage (0–100) to `pumps.<deviceNumber>.control.speed`. The adapter sends the command, confirms it, and re-reads the pump shortly after so the states reflect reality.

Read telemetry — the values under `telemetry` update live on every poll (fast values like power and rpm each cycle, slower ones like temperature every few cycles to keep the cloud happy). Use them in VIS, charts, or scripts.

Example (JavaScript adapter):

```
// Run the "Waterfall" pump at 70 %
setState('pondpump.0.pumps.1234567.control.speed', 70);

// Log its live power
on('pondpump.0.pumps.1234567.telemetry.power', (obj) => {
    log('Pump power: ' + obj.state.val + ' W');
});
```

---

8. Local mode (in-house LAN)

The adapter can run **entirely over your local network**, without the internet. Set **Connection mode** to `local` and it will:

- start a small **TLS server** and send a **UDP wake** packet to the controller,
- the controller **connects back** over TLS and authenticates with the **device password**,
- then the adapter reads the gateway + pumps and polls **live telemetry** (power, speed, temperature, voltage), and lets you **switch on/off and set the speed** — all over the LAN.

What you need:

- **Controller IP** — the EGC controller's address in your network.
- **Device password** — the 64-character value (see chapter 4.4 for how to read it).
- an open network path: **UDP 5959** out to the controller and **TCP 5999** back to ioBroker. If the controller and ioBroker are on different subnets/VLANs, allow those two directions.

Leave **Bind address** at `0.0.0.0` — the adapter auto-detects the host address the controller should connect back to.

> 💡 **Note:** the current **speed setpoint** (the % value) is not read back over the local channel — the `control.speed` state reflects the last value you set from ioBroker. **On/off is read live** (derived from the pump's power draw), so it also reflects changes you make in the OASE app.

---

9. vis-2 widgets

The adapter ships **three ready-made vis-2 widgets** — there is nothing extra to install. As soon as the adapter is installed, vis-2 restarts automatically and the widgets appear in the vis-2 editor under the widget group **"Pond Pump"**.

> 💡 **Tip:** If the widgets don't show up in the editor right after installation, reload the editor page in your browser once (**Ctrl + F5**).

9.1 Adding a widget and linking it to a pump

1. Open the **vis-2 editor** and drag one of the two widgets from the **"Pond Pump"** group onto your view.
2. On the right, in the widget settings, pick your `pondpump` instance under **Instance** (e.g. `pondpump.0`).
3. Below it, pick the pump under **Pump** — the list automatically shows every detected pump by name.

That's all you need to configure: the widgets know the matching object IDs themselves and connect to the selected pump automatically.

9.2 The "Pump visualization" widget (PumpVisual)

This widget shows the pump graphically:

- An **impeller** spins depending on the pump speed — in 10 % steps, from a slow crawl to fast.
- When the pump is **off**, the impeller stands still with a **red cross** over it.
- When **Seasonal Flow Control (SFC)** is active, a rotating **ice crystal** replaces the impeller.
- Below the graphic are the live values: **power** (W), **speed** (rpm) and **Power** (the setpoint in %).
- When a **water temperature** is available (state `telemetry.waterTemperature`), the impeller shifts left and a **filled thermometer** with the reading appears on the right; the fill is colour-coded by temperature (cold blue → warm amber). Without a value the display is unchanged.

A coloured badge in the top-right shows the state: **Running**, **Off** or **Seasonal mode**.

9.3 The "Pump control" widget (PumpControl)

This widget controls the pump:

- **On / Off** switches the pump.
- The **slider** sets the power in percent. The command is only sent when you release the slider, so dragging never floods the device with commands. The **quick buttons** (0/25/50/75/100 %) set the value directly.
- **Seasonal Flow Control (SFC)** can be switched on/off with the button. The button reacts instantly (with a spinner until the device confirms) — no need to press it repeatedly.

> 💡 **What is SFC?** "Seasonal Flow Control" is OASE's temperature-dependent seasonal flow regulation: when SFC is active the pump automatically reduces its flow rate and delivery head (by up to −50 %), adapting to the pond biology over the year. It is **not** frost protection.

9.4 The "Scheduler status" widget (PumpScheduler)

This widget shows at a glance **what the built-in schedule/rule scheduler** (chapters 10–11) is doing with the pump right now — and **why**:

- A **status badge**: **Scheduler active**, **Manual / off** (no valid schedule) or **Fail-safe** (sensor loss).
- The current **output** in % shown large — next to a **small impeller** that spins (optionally animated) with the real speed — plus the running state (Running/Off/Seasonal mode), **day/night**, and the scheduler's **target power**.
- **Reason chips**: where the base comes from (**temperature curve**, **time window** or **base power**) and which modifiers apply right now (**night protection**, **weather boost**, **frost hold**).
- The **active window**, the **next change** time and the pump's **sunrise/sunset**.
- **Actuators:** if the schedule has **actuator windows** (waterfall, stream, aerator, …), each one is listed **above the telemetry**, one row each in the order **icon — name — impeller**. Set the name and icon in the schedule editor (mode "Actuator"); without a name it shows "Actuator 1", "Actuator 2" … The small **light-green impeller** spins (optionally animated) while the actuator is **on** and stands still (dimmed) while off. In the widget settings (**"Actuators"** section) you can show or hide **each actuator individually**, and choose the **wheel colour** for the on and off states (defaults match the previous look).
- **Water temperature** (colour-coded), **power** (W) and **speed** (rpm).
- A **control bar** with the basic functions (on/off, quick power, SFC). A hint reminds you that the scheduler may re-apply its target on the next run.

The values come from new read-only `pumps.<n>.schedule.*` states the scheduler keeps up to date on every evaluation — you can also use them in your own scripts or in history.

9.5 Adjusting the appearance

In the widget settings under **Appearance** you can, among other things, choose the **accent colour**, hide the **card background**, turn off the **animation**, or show/hide individual parts (values, on/off buttons, quick buttons, SFC, control bar, telemetry).

10. Schedules (running pumps on a timetable)

The adapter can run each pump on a **daily schedule** instead of a fixed setting. You define time windows that each set a **power %** or switch **SFC** on/off; outside every window the pump falls back to a configurable **base power**.

10.1 Turn on scheduling for a pump

1. Open the instance settings and stay on the **Connection** tab.
2. Scroll to the **Schedules** section at the bottom. It lists every pump the adapter has discovered. (If it is empty, let the adapter run once so it finds the pumps, then reload the page.)
3. Switch on the pump(s) you want to run on a timetable. Each enabled pump gets its own **“Scheduler – <pump name>”** tab at the top.

10.2 Define the time windows

Open the pump's **Scheduler** tab:

- **Base power %** — applied whenever no window is active (e.g. a quiet night level).
- The table holds the **time windows**. Add a row with **Add schedule** and set:
  - **Start** / **End** — daily times (HH:MM). A window may not cross midnight — split it into two.
  - **Mode** — **Power %** (the window sets a fixed power), **SFC** (the window switches Seasonal Flow Control on or off), or **Actuator** (the window drives an **external state**, e.g. a waterfall/UVC — combine it with astro bounds, see 10.4).
  - **Value** — the power percentage, or on/off for SFC; for an **Actuator**, the **target object id** plus an on-value (active) and an optional off-value (inactive; blank = leave it untouched outside).
- Windows **must not overlap.** The editor validates live and shows a red message if two windows collide; the adapter also re-checks before applying, so an invalid schedule is never run.

Remember to **Save**.

10.3 How it runs

The adapter evaluates the schedule and applies the target **at each window boundary** (and once at start-up):

- Inside a **Power** window it sets that power and switches SFC off.
- Inside an **SFC** window it switches SFC to the chosen state and keeps the base power (which only matters while SFC is off).
- Outside every window it applies the **base power** with SFC off.

It only writes when the target actually changes, so scheduling coexists with manual control: your last manual change stays until the next window boundary moves the pump again.

10.4 Astronomical windows (sunrise/sunset)

A window's start and end need not be a fixed clock time. For each boundary you can choose **Sunrise** or **Sunset** instead of **Clock time** and give an **offset in minutes** (may be negative). Examples: start = "Sunset + 0", end = "Sunrise + 120" is a **night window** that wraps past midnight; "Sunrise − 30" starts half an hour before sunrise. The sun times are recomputed daily.

> 💡 **Note (research):** a night-time **reduction of the flow is counter-productive in summer** (the oxygen minimum is at night). Astro windows are best used as a **protection window** (don't reduce) and for side actuators (waterfall). See `doc/research/` .

Location: the sun times need a location. On the **Connection** tab, in the **Schedules** section, pick the **location mode**:

- **Use the ioBroker system location** (default) — takes latitude/longitude from the ioBroker system settings.
- **One location for all pumps** — a shared position; set it on the **map** (click or drag the marker), by **address search**, or in the latitude/longitude fields.
- **A location per pump** — each pump sets its own position on its own tab.

Night protection: in the **Temperature / weather control** section you can enable **night protection** per pump. The flow is then **not reduced below a floor** (default 100 %) **during the astronomical night**, as long as the water temperature is at/above a threshold (default 18 °C) — exactly what the research recommends (the oxygen minimum is at night; a warm-night reduction is harmful). It needs a location and is still capped by the **maximum power**.

11. Temperature- and weather-based control

On top of fixed time windows, each pump can be driven by **water temperature and weather**. The idea comes from the pond-flow research (see `doc/research/`): the **flow follows the water temperature** (colder water = less circulation, warmer = more), and **weather may only raise the flow**, never lower it (e.g. heat → more aeration).

> 💡 **Important:** the pump state `telemetry.temperature` is the **device** temperature, not the water. Use a **real water sensor** as the curve source (e.g. a pond probe from another adapter).

Open the pump's **Scheduler** tab and scroll to the **Temperature / weather control** section.

11.1 How it relates to the time windows

The select at the top decides how the curve relates to the time windows:

- **Curve overrides the active time window** — the water-temperature curve always sets the power.
- **Curve applies only outside the time windows** — inside a window the window wins; outside, the curve takes over.

11.2 Water temperature sensor and curve

At the top, pick the **water temperature sensor**: the dropdown lists the pump's own temperature sensors **with their current value** — compare them against a thermometer you trust and pick the one that reads the water. Your choice feeds the new `telemetry.waterTemperature` state and **pre-fills the curve source**. For an **external** probe, leave this on "none" and enter its object as the curve source below (magnifier icon). Either way, `telemetry.waterTemperature` mirrors the **effective curve source** — so it shows your external sensor's value too, not only an on-device sensor.

To turn on the curve:

1. Turn on **Water temperature → power curve**.
2. Check the **water-temperature source** — it is pre-filled from the sensor above; for an external probe, pick its object with the **magnifier icon**.
3. Enter **points** (temperature °C → power %). Values are interpolated linearly between points and clamped beyond the ends. **Load default curve** fills in the recommended research curve (Q10 rule, 17 °C → 100 %) as a starting point.

Safety behaviour: if the sensor drops out (the source has no value), the pump runs at **100 %** to be safe — too much circulation only costs electricity, too little costs fish.

11.3 Fine-tuning (optional)

Below the curve are optional limits. In the admin UI each field shows a **suggested value** (grey placeholder) and a **help text**:

- **Minimum power %** — a lower limit; the flow never drops below it. Suggested **35–40 %**.
- **Maximum power %** — an upper limit; the flow never exceeds it (not even on boost or fail-safe). E.g. **90 %** for a pump that only runs that high.
- **Temperature smoothing (hours)** — averages the temperature so short spikes don't constantly re-adjust the pump. Suggested **12–24**, `0` = off.
- **Hysteresis (°C)** — the curve is only re-mapped after the temperature has moved by this much. Suggested **0.5–1**, `0` = off.
- **Max. change (% per hour)** — limits how fast the power may change (a gentle ramp). Suggested **10–20**, `0` = instant.

11.4 Weather rules

The rules table below can additionally **raise** the flow or drive actuators. Each rule compares a **source** (any state id, e.g. an outdoor temperature or a rain OID) against a **threshold** with an operator (`<` , `≤` , `>` , `≥` , `=` , `≠`). When it matches, its **effect** applies:

- **Raise to power %** — raises the power to at least this value (never lowers).
- **Boost to 100 %** — full power (e.g. heat).

- **Hold (frost)** — freezes the power at its last value.
- **SFC on/off** — switches the pump's Seasonal Flow Control.
- **Set actuator** — writes any **target object id** to a value (true/false or a number), e.g. turning an aerator or waterfall on.

All matching rules combine (raises take the maximum; a "hold" freezes unless a raise wins; actuator writes accumulate). The adapter subscribes to the source states and re-evaluates **the moment they change**, not just at window boundaries.

> 💡 **Note on SFC:** if the curve is regulating power while the pump's **native SFC** is on, the pump overrides the setpoint — the adapter then warns in the log. Turn SFC off, or drive it via an "SFC" rule.
>
> **Upgrading from 0.3.0:** the rule model changed. Rules from 0.3.0 (effects *Power %/SFC/Off*) are inert and must be recreated with the new effects.

Don't forget to **save**.

12. Troubleshooting

Symptom	What to check
`info.connection` stays **false**	Is a **refresh token** entered? Capture a fresh one (chapter 4) — tokens can expire if you log in elsewhere.
Log says **AUTH FAILED**	The refresh token is invalid/expired → capture a new one.
No pumps appear	Are the pumps online in the **OASE app**? The adapter mirrors the cloud inventory.
Commands do nothing	Wait for the **first successful poll** (the adapter learns the pump addressing then). Check the log.
Want more detail	Set the instance **log level to** `debug` — every step is logged with a tag like `[poll]`, `[cloud/auth]`, `[cloud/cmd]`, `[schedule]`, `[astro]`, `[geocode]`. Secrets are never logged.
A pump runs at an unexpected power	On `debug`, each scheduler tick logs the **full decision chain** for the pump — see below.

The log lines are tagged by component so any problem can be pinpointed. When reporting an issue, include the debug log around the failure.

Reading the scheduler decision log

On `debug` every scheduler evaluation prints, per pump, exactly why it chose the power/SFC it did:

- a **tick** line with the current time and all raw source values,
- an **inputs** line: raw / smoothed / mapped water temperature (with the smoothing τ and hysteresis K), the resolved sunrise/sunset and day/night, and the priority / min / max power,
- a **decision** line: the base (from the curve or the active window), the Q_min floor, night protection, every weather rule that matched, the actuator windows and the Q_max ceiling, ending in the final power/SFC,
- the **ramp/hold** state and the **next re-evaluation** time.

So a single `[schedule] pump 1 decision: …` line tells you the complete reasoning — no guessing why a pump sits at a given percentage.

---

13. Privacy & safety

- Your **OASE account password** is never entered into or stored by the adapter.
- The **refresh token** and **device password** are stored **encrypted** in ioBroker.
- The adapter only talks to the OASE cloud (or, in local mode, directly to your controller).
- Use at your own risk — this is an unofficial community project, not affiliated with OASE GmbH.

Questions or problems? Open an issue at the project's GitHub repository. Happy pond-keeping! 🐟